

# 大模型并行策略全解：DP、FSDP、TP、PP/VPP、CP、EP 与多维组合

**原始问题：**详细讲解不同的并行策略：PP,EP,DP,CP,TP等，每一种都讲透彻，并且列出各自的异同，适合的场景，具体应用例子，然后全部串起来，举一个简单的数据例子，把各种并行策略都叠加使用，完整的展现每种并行策略的实际应用。

提问：2026/07/29 20:49    更新：2026/07/29 20:49    重要度：5/5

## 大模型并行策略全解：DP、FSDP、TP、PP/VPP、CP、EP 与多维组合

### 一句话结论

这些并行策略并不是互相替代，而是在切分不同维度：

DP / DDP：切 Batch，复制模型  
FSDP / ZeRO：在 DP 语义下进一步切参数、梯度和优化器状态  
TP：切单层内部的矩阵、Hidden 或 Attention Heads  
PP：切模型深度，把不同 Layers 放到不同 Stage  
VPP：把每个 PP Stage 再切成多个 Chunks，优化流水线调度  
CP：切 Sequence / Context，让每张 GPU 只保存一部分长序列激活  
EP：切 MoE Experts，把不同专家放到不同 GPU

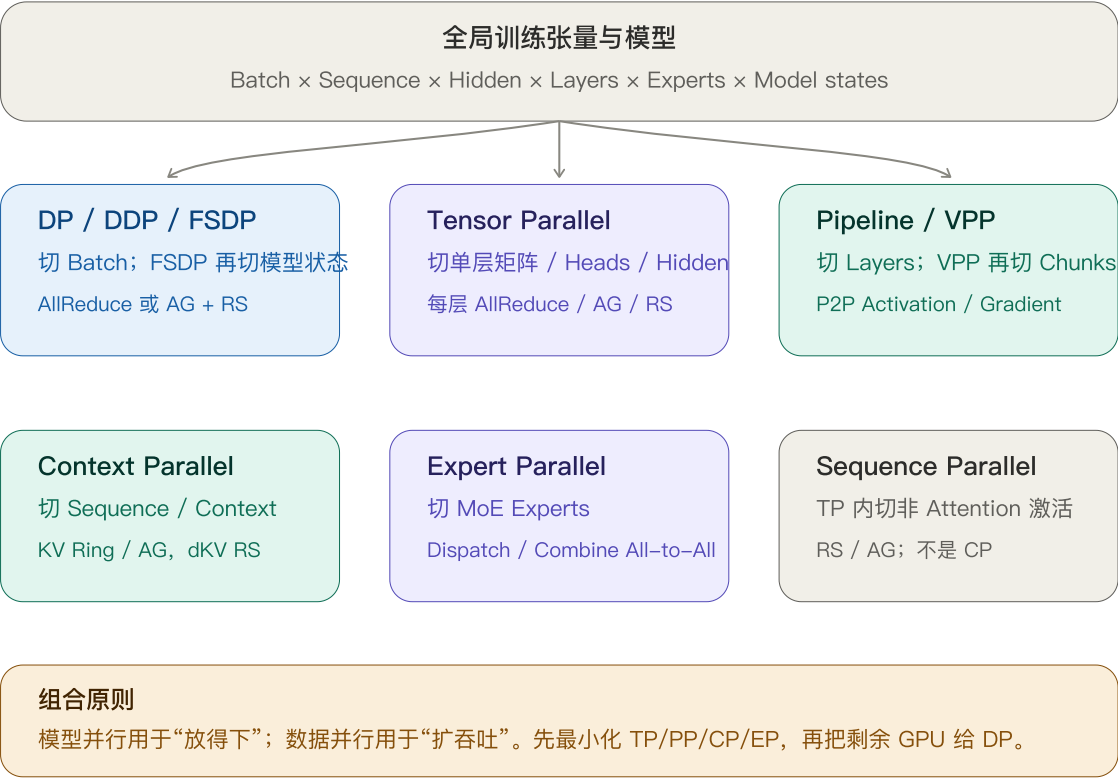
选择原则：

模型并行主要解决“单卡放不下”；数据并行主要解决“吞吐扩展”。先使用尽可能小的 TP/PP/CP/EP 让模型和激活放得下，再把剩余 GPU 尽量用于 DP。

所有并行都会降低某类显存或扩大可训练规模，但同时引入通信：

内存节省不是免费午餐  
= 更复杂的数据布局  
+ 更多通信  
+ 更严格的拓扑映射  
+ 更难的调试与 Checkpoint

同一个训练问题，可以沿不同独立维度切分



---

## 一、先建立统一坐标系：每种并行到底切什么

假设一次 Transformer 训练的核心对象包括：

数据: [B, S]  
激活: [B, S, H]  
Attention: [B, S, Heads, HeadDim]  
模型深度: L Layers  
MoE Experts: E  
训练状态: 参数、梯度、优化器状态

符号：

B      Batch Size  
S      Sequence Length  
H      Hidden Size  
L      Transformer Layer 数  
E      Expert 数  
TP      Tensor Parallel Size  
PP      Pipeline Parallel Size  
CP      Context Parallel Size  
EP      Expert Parallel Size  
DP      Data Parallel Size

并行维度总览：

| 策略        | 主要切分对象             | 每卡主要保留什么            | 主要通信                                         | 最直接解决的问题           |
|-----------|--------------------|---------------------|----------------------------------------------|--------------------|
| DP/DDP    | Batch              | 完整模型副本 + 本地 Batch   | Gradient AllReduce                           | 扩大数据吞吐             |
| FSDP/ZeRO | Batch + 模型状态       | 参数/梯度/优化器状态分片       | Parameter AllGather + Gradient ReduceScatter | 训练状态放不下            |
| TP        | Layer 内 Tensor     | 矩阵/Heads/Hidden 的分片 | 每层 AllReduce/AllGather/ReduceScatter         | 单层或大矩阵放不下          |
| PP        | Layers / 深度        | 一段连续 Layers         | P2P Activation/Gradient                      | 模型层数多、整体放不下        |
| VPP       | PP Stage 内 Chunks  | 多个不连续 Layer Chunks  | 更多 P2P                                       | 减少 Pipeline Bubble |
| CP        | Sequence / Context | 一段 Token 与激活        | KV Ring/AllGather, dKV ReduceScatter         | 长上下文激活 OOM         |
| EP        | MoE Experts        | 一部分 Experts         | Dispatch/Combine All-to-All                  | MoE Expert 参数放不下   |

另外还有 **Sequence Parallelism (SP)**：它通常依附 TP，只切 LayerNorm、Dropout 等非 Attention 区域的 Sequence 激活；CP 则贯穿网络输入和全部激活，并专门解决 Attention 的跨 Token 依赖。SP 不是 CP 的同义词。

---

## 二、Data Parallelism：DP、DDP 与 FSDP

### 2.1 普通 DP/DDP 的核心

Data Parallel 的逻辑是：

每个 DP Rank 持有一份相同模型  
每个 Rank 处理不同的数据子 Batch  
Backward 后同步梯度  
所有 Rank 用相同全局梯度更新参数

设：

Global Batch = 8  
DP = 4  
Gradient Accumulation = 1

则每个 DP Replica 的 Local Batch:

```
B_local = B_global ÷ DP
          = 8 ÷ 4
          = 2
```

四个 Rank:

```
DP0: 样本 0,1
DP1: 样本 2,3
DP2: 样本 4,5
DP3: 样本 6,7
```

各自计算本地梯度:

```
g_0, g_1, g_2, g_3
```

然后执行 Gradient AllReduce:

```
g_global = (g_0 + g_1 + g_2 + g_3) ÷ 4
```

所有 Rank 都得到相同的 `g_global`，因此参数保持一致。

## Global Batch 公式

```
Global Batch
= Micro Batch Per GPU
× Gradient Accumulation Steps
× DP Size
```

例如:

```
Micro Batch = 2
Accumulation = 4
DP = 8

Global Batch = 2 × 4 × 8 = 64
```

TP、PP、CP、EP 通常共同组成一个 Model Replica，它们不会像 DP 一样自动扩大独立样本数。

## 2.2 DDP 的通信与 Overlap

PyTorch DDP 通常将梯度分 Bucket:

```
后层梯度先计算完成
→ 对该 Bucket 异步 AllReduce
```

因此 DDP 的 Gradient AllReduce 很适合与 Backward Overlap。

通信发生在每个 Step 的 Backward 阶段，频率比 TP 的“每层通信”低，因此 DP 往往更适合跨节点扩展。

## 2.3 普通 DDP 的显存问题

每个 DP Rank 都保存完整：

- 参数；
- 梯度；
- Optimizer States；
- 混合精度的 Master Weights；
- 本地 Activation。

假设混合精度 Adam 经典估算约为：

```
FP16/BF16 参数      2 Byte
FP16/BF16 梯度      2 Byte
FP32 Master Weight  4 Byte
FP32 Adam m         4 Byte
FP32 Adam v         4 Byte
-----
合计约              16 Byte/参数
```

一个 10B 模型仅训练状态约：

```
10×109 × 16 Byte = 160 GB
```

普通 DDP 在每张卡上复制这 160GB，无法训练。

## 2.4 FSDP / ZeRO：DP 语义下分片模型状态

FSDP 保留“不同 Rank 处理不同数据”的 DP 语义，但不再让每卡永久保存完整训练状态。

| 策略                          | 分片内容                | 对应 ZeRO |
|-----------------------------|---------------------|---------|
| Shard Optimizer             | Optimizer States    | ZeRO-1  |
| Shard Gradients + Optimizer | Gradients、Optimizer | ZeRO-2  |
|                             |                     |         |

| 策略                                       | 分片内容   | 对应 ZeRO             |
|------------------------------------------|--------|---------------------|
| Shard Parameters + Gradients + Optimizer | 全部训练状态 | ZeRO-3 / Full Shard |

Full Shard 中：

平时：每卡只保存 1/DP 参数分片  
计算某个 FSDP Unit 前：Parameter AllGather  
Backward 后：Gradient ReduceScatter  
Optimizer：每卡只更新自己的状态分片

理想每卡训练状态：

$M\_state\_per\_rank \approx M\_total\_state \div DP$

10B 模型、DP=8：

$160\text{ GB} \div 8 = 20\text{ GB/Rank}$

注意这不包含：

- Activation；
- AllGather 临时完整参数峰值；
- Communication Buffer；
- CUDA Workspace；
- Fragmentation。

## 2.5 DP/FSDP 适用场景

适合普通 DDP：

- 模型完整训练状态能放入单卡；
- 主要目标是扩大吞吐；
- 希望实现简单、计算效率高；
- 可以使用大 DP Group。

适合 FSDP/ZeRO：

- 模型训练状态超过单卡；
- 单个 Layer 仍可在单卡或 TP Group 内计算；

- 有足够通信带宽；
- 可以通过 Prefetch/Overlap 隐藏 AllGather/ReduceScatter。

不解决：

- 单个巨大 Layer 本身放不下，仍需 TP；
- 长序列 Activation OOM，仍需 CP/Checkpoint；
- 模型太深的 Pipeline 显存，仍可用 PP。

---

## 三、Tensor Parallelism (TP)

### 3.1 TP 切的是什么

TP 将一个 Layer 内部的大矩阵乘切到多张 GPU。

例如 Linear：

```
Y = XW  
  
X: [T, H]  
W: [H, F]  
Y: [T, F]
```

设：

```
H = 16  
F = 64  
TP = 2
```

#### Column Parallel

沿输出维 **F** 切权重：

```
W = [W_0 | W_1]  
W_0: [16, 32]  
W_1: [16, 32]
```

每个 TP Rank：

```
Y_0 = XW_0 → [T, 32]
```

```
Y_1 = XW_1 → [T,32]
```

逻辑完整输出：

```
Y = concat(Y_0,Y_1) → [T,64]
```

如果下一算子能直接消费分片输出，可以不立刻 AllGather。

## Row Parallel

沿输入维切：

```
X = [X_0 | X_1]  
W_0: [32,H_out]  
W_1: [32,H_out]
```

各 Rank 得到部分和：

```
Z_0 = X_0W_0  
Z_1 = X_1W_1
```

完整结果：

```
Z = Z_0 + Z_1
```

需要 AllReduce，或在 Sequence Parallel 中使用 ReduceScatter 保持激活分片。

## 3.2 Transformer 中的典型 TP

### Attention

设：

```
Q Heads = 4  
HeadDim = 4  
H = 16  
TP = 2
```

每个 TP Rank 计算：

```
2 个 Q Heads
```

若 KV Heads=2，也可每 Rank 持有 1 个 KV Head。



各 Rank 独立计算部分 Heads，然后输出投影通过 Row Parallel 归约。

## MLP

```
W1: [16, 64] → 每 Rank [16, 32]  
W2: [64, 16] → 每 Rank [32, 16]
```

典型安排：

```
Column Parallel W1  
→ Activation 本地执行  
→ Row Parallel W2  
→ AllReduce / ReduceScatter
```

这样中间非线性之前不需要额外通信。

## 3.3 TP 的收益

- 单层参数约降到  $1/TP$ ；
- 大矩阵计算量分到多卡；
- Attention Heads 可以分片；
- 配合 SP 可降低部分 Activation；
- 单层超过单卡显存时不可替代。

## 3.4 TP 的代价

TP 通信发生在几乎每个 Transformer Layer，且下一计算常立即依赖通信结果，难以完全隐藏。

因此：

TP 通信频率最高、依赖最紧，优先放在单机 NVLink/NVSwitch 域内。

TP 过大还会导致：

- 每卡 GEMM 变小，Tensor Core 利用率下降；
- Collective Latency 占比上升；
- Hidden/Head/FFN 维度整除限制；
- Kernel Launch 与通信难 Overlap。

## 3.5 TP 适用场景

- 单个 Layer、Embedding 或 FFN 矩阵放不进单卡；
- Hidden Size 很大；
- Attention Heads 能整除 TP；
- 同一节点内有高速互联；
- 与 PP/DP 组合训练大模型。

实际例子：一个大模型的 64 个 Attention Heads 使用 TP=8，每卡计算 8 个 Heads；FFN 中间维 32768 每卡持有 4096。

---

## 四、Pipeline Parallelism (PP) 与 VPP

### 4.1 PP 切模型深度

假设 8 层 Transformer：

```
PP = 2
Stage 0: Layers 0-3
Stage 1: Layers 4-7
```

Stage 0 Forward 输出 Activation，通过 P2P 发送给 Stage 1：

```
[B_local, S, H]
```

Backward 时 Stage 1 将 Activation Gradient 发回 Stage 0。

### 4.2 Microbatch 与流水线

不拆 Microbatch 时：

```
Stage 0 完成整个 Batch
→ Stage 1 才开始
```

GPU 利用率低。

将 Batch 拆成  $m$  个 Microbatches 后，不同 Stage 同时处理不同 Microbatch：

```
Stage 0: MB3 Forward
Stage 1: MB2 Forward / 其他 MB Backward
```

1F1B 调度分为：

```
Warm-up → Steady 1F1B → Cool-down
```

普通平衡 PP 的经典 Bubble：

```
T_bubble = (PP-1)(t_f+t_b)
```

相对 Useful：

```
Bubble / Useful = (PP-1) ÷ m
```

Bubble 占实际总时间：

```
Bubble / Total = (PP-1) ÷ (m+PP-1)
```

## 4.3 VPP

VPP 将每张物理 Stage 再切成多个 Model Chunks，使同一 GPU 扮演多个虚拟 Stage。

若每卡  $v$  个 Chunks：

```
T_bubble_vpp ≈ (PP-1)(t_f+t_b) ÷ v
```

收益：减少 Bubble。

代价：

- P2P 边界增加；
- Pipeline 通信量约增  $v$  倍；
- 调度和 Activation 管理更复杂；
- Chunk 太细会增加 Kernel/P2P Launch 开销。

## 4.4 PP 适用场景

- 模型 Layer 很多，单个 Stage 装不下全模型；
- 希望跨节点扩展，而不扩大高频 TP Group；
- Stage 可以做到计算均衡；
- 有足够 Microbatch 填充流水线。

不适合：

- 模型层数少；
- Global Batch 太小，Microbatch 数不足；
- Stage 严重不均衡；
- Activation 边界特别大且网络慢。

实际例子：96 层模型用 PP=8，每 Stage 12 层；跨节点主要传 Activation，而把 TP 限制在单机内。

---

## 五、Context Parallelism (CP)

### 5.1 CP 切 Sequence

输入：

```
X: [B,S,H]
```

设：

```
B = 2  
S = 8  
H = 16  
CP = 2
```

每个 CP Rank 保存：

```
X_local: [2,4,16]
```

除 Attention 外，Linear、LayerNorm、MLP 通常按本地 Token 独立工作，不需要看到其他 Sequence Chunk。

## 5.2 为什么 Attention 特殊

本地 Query 只对应 **S/CP** 个 Token，但每个 Query 需要关注完整序列的 Keys/Values：

```
Q_local: [B,S/CP,Heads,HeadDim]
K/V_global: [B,S,KVHeads,HeadDim]
```

所以 CP 需要交换 KV：

- 概念上可看作 KV AllGather；
- 高性能实现常将其转成 Ring P2P，边收 KV Chunk 边计算 Attention；
- Backward 对 dK/dV 使用对应 ReduceScatter 或 Ring 归约。

每卡仅长期保存本地 KV Chunk，Backward 需要时可重新 Gather/交换，以降低 Activation Memory。

## 5.3 CP 与 SP 区别

| 维度        | Sequence Parallel (SP)     | Context Parallel (CP) |
|-----------|----------------------------|-----------------------|
| 所属        | 通常依附 TP                    | 独立 CP Process Group   |
| 切分范围      | LayerNorm/Dropout 等非 TP 区域 | 网络输入和几乎全部激活           |
| Attention | 通常仍依赖 TP 布局                | 专门处理跨 Context KV 依赖   |
| 主要目的      | 减少 TP 重复激活                 | 长上下文 Activation OOM   |
| 是否需要 TP>1 | 通常需要                       | 可以 TP=1               |

## 5.4 CP 收益与代价

收益：

```
Activation Memory ≈ 原来的 1/CP
Local Token Compute ≈ 原来的 1/CP
```

代价：

- 每个 Attention Layer 都要交换 KV；
- Causal Attention 需要负载均衡，避免某些 Rank 计算更多有效块；
- CP 太大时每 Rank Q Chunk 太小；
- 通信与 FlashAttention 计算的 Overlap 变难；
- Sequence Length 必须满足分片/打包约束。

GQA/MQA 的 KV Heads 少，因此可显著降低 CP 的 KV 通信量。

## 5.5 CP 适用场景

- 长上下文，例如 8K、32K、128K；
- Activation Memory 成为主瓶颈；
- 不希望继续增大 TP 或使用昂贵 Full Recomputation；
- Attention Kernel 支持 CP/Ring 计算；
- 能与 TP、PP、DP 组合。

实际例子：S=128K、CP=8，每 Rank 长期处理约 16K Query Tokens，并按 Ring 流式接收其他 KV Chunks。

---

## 六、Expert Parallelism (EP)

### 6.1 EP 切 Experts

MoE Layer 有：

```
E = 4 Experts
EP = 2
```

每个 EP Rank 保存：

```
E_local = E ÷ EP = 2 Experts
```

Router 可把任意 Token 路由到任意 Expert，所以 Token 必须移动到 Expert 所在 Rank。

## 6.2 两次 All-to-All

Forward:

```
Router Top-k
→ Permute Tokens by Expert
→ Dispatch All-to-All
→ Local Expert / Grouped GEMM
→ Combine All-to-All
→ Unpermute + Weighted Sum
```

Top-k=2 时，每个 Token 产生两个 Token-Expert Assignments。

## 6.3 EP 收益

- Expert 参数显存约降到  $1/EP$ ；
- 可扩展到大量 Experts；
- 相比在单个 Expert 内增大 TP，EP 往往能保持更大的本地 GEMM；
- 适合 MoE 架构。

## 6.4 EP 代价

- 每个 MoE Layer 两次 All-to-All；
- Router 负载不均导致 Rank 热点；
- 每 Expert Token 数动态变化；
- EP 太大时每 Expert GEMM 太小；
- 跨节点 All-to-All 昂贵；
- 需要 Grouped GEMM、Permutation Fusion、负载均衡和专用 Dispatcher。

当 EP 与 TP 组合时，Megatron 类实现通常要求启用 Sequence Parallel，以保证 Token/Hidden 布局可组合。

## 6.5 EP 适用场景

- 模型存在 MoE Layers；
- Expert 总参数超过单卡；

- All-to-All 网络足够快；
- 每 Expert 能获得足够多 Token；
- 有成熟的 Router 与 Grouped GEMM。

实际例子：64 Experts、EP=8，每 Rank 保存 8 Experts；Top-2 Token 经 All-to-All 到对应 Rank。

---

## 七、六类并行的完整异同比较

| 维度            | DP/DDP        | FSDP/ZeRO          | TP             | PP/VPP               | CP                  | EP              |
|---------------|---------------|--------------------|----------------|----------------------|---------------------|-----------------|
| 切分对象          | Batch         | Batch + 模型状态       | Layer 内 Tensor | Layers/Chunks        | Sequence            | Experts         |
| 是否复制模型参数      | 是             | 分片                 | 分片             | 每 Stage 只持有部分 Layers | 通常复制当前模型分片          | 只持有部分 Experts   |
| 每卡计算量         | 约不变           | 约不变                | 约 1/TP         | 约 1/PP               | 约 1/CP，Attention含通信 | 只算路由到本地 Experts |
| 参数显存          | 不降            | 约 1/DP             | 约 1/TP         | 约 1/PP               | 通常不降                | Expert 参数约 1/EP |
| Activation 显存 | Local Batch   | Local Batch        | SP时部分约 1/TP    | Stage内；VPP可能更复杂      | 约 1/CP              | 动态，依路由          |
| 主要通信          | Gradient AR   | Param AG + Grad RS | 每层 AR/AG/RS    | P2P Activation       | KV Ring/AG + dKV RS | 两次 A2A          |
| 通信频率          | 每 Step/Bucket | 每Unit前后向           | 每Layer多次       | 每Microbatch边界        | 每Attention Layer    | 每MoE Layer      |
| Overlap难度     | 较容易           | 可Prefetch          | 较难             | 依调度                  | 与Attention计算流水      | 复杂且动态           |
| 最适合链路         | 可跨节点          | 高带宽更好              | 机内NVLink       | 可跨节点                 | 高带宽域优先              | NVLink/高速网络     |
| 核心目标          | 扩吞吐           | 省训练状态显存            | 切高层            | 切深模型                 | 长上下文                | 切MoE专家          |
| 主要风险          | 状态复制          | 通信/峰值              | 小GEMM、高频通信     | Bubble/不均衡           | KV通信                | A2A/负载不均        |

口诀：

Batch 太大要扩吞吐 → DP  
训练状态放不下 → FSDP  
单层放不下 → TP  
层数太多/跨节点扩模型 → PP  
长序列 Activation OOM → CP  
MoE Experts 放不下 → EP



## 八、统一组合例子：64 卡五维并行

### 8.1 模型与数据

使用一个教学型 MoE Transformer：

```
World Size = 64 GPUs
Layers L = 8
Hidden H = 16
Q Heads = 4
KV Heads = 2
HeadDim = 4
Dense FFN F = 64
MoE Experts E = 4
MoE Top-k = 2
Global Batch = 8
Sequence Length S = 8
Gradient Accumulation = 1
```

并行配置：

```
TP = 2
PP = 2
CP = 2
EP = 2
DP = 4
```

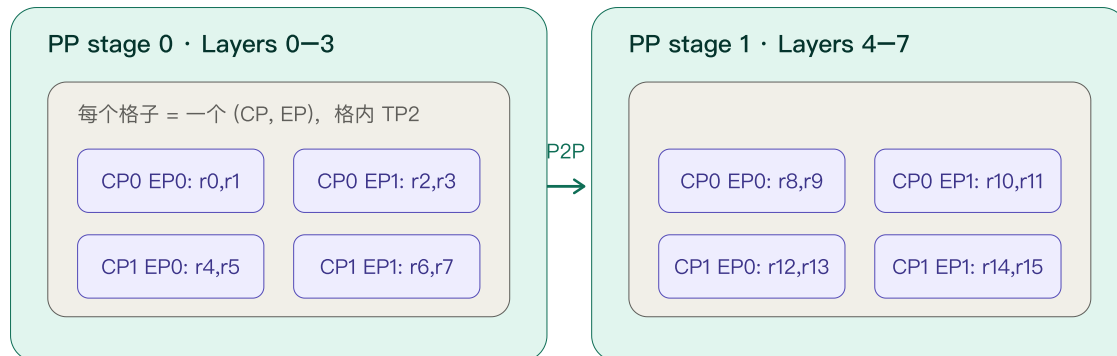
World Size：

```
2 × 2 × 2 × 2 × 4 = 64
```

World size = TP2 × PP2 × CP2 × EP2 × DP4 = 64 GPUs



展开 DP replica 0: 16 GPUs



Rank 坐标（教学用，TP 最快变化）

$$\text{rank} = (((dp \times 2 + pp) \times 2 + cp) \times 2 + ep) \times 2 + tp$$

以 rank 0=(dp0,pp0,cp0,ep0,tp0) 为例：

TP {0,1} · EP {0,2} · CP {0,4} · PP {0,8} · DP {0,16,32,48}

真实框架的 Rank 顺序可配置；不变的是每种 Process Group 只改变一个坐标。

每个 DP Replica 需要完整覆盖一个逻辑模型，因此占用：

```
TP × PP × CP × EP
= 2 × 2 × 2 × 2
= 16 GPUs
```

共有 4 个 DP Replicas：

```
Replica 0: ranks 0-15
Replica 1: ranks 16-31
Replica 2: ranks 32-47
Replica 3: ranks 48-63
```

## 8.2 Rank 五维坐标

教学用 Rank 排列，TP 最快变化：

```
rank = (((dp × PP + pp) × CP + cp) × EP + ep) × TP + tp
```

代入各维 Size=2/4：

```
rank = (((dp×2 + pp)×2 + cp)×2 + ep)×2 + tp)
```

Rank 0 坐标:

```
(dp=0, pp=0, cp=0, ep=0, tp=0)
```

固定其他坐标，只改变一个维度，得到 Rank 0 所属通信组：

```
TP Group: {0,1}
EP Group: {0,2}
CP Group: {0,4}
PP Group: {0,8}
DP Group: {0,16,32,48}
```

真实框架 Rank Ordering 可以配置，例如 `tp-cp-ep-dp-pp`；不变的是：每个 Process Group 只沿一个逻辑坐标变化。

## 8.3 PP 切 Layers

```
PP Stage 0: Layers 0-3
PP Stage 1: Layers 4-7
```

每个 Stage 内部再由  $TP \times CP \times EP = 8$  张 GPU 协作。

若启用 VPP=2，可进一步变成：

```
GPU Group for Stage 0: Chunk 0 + Chunk 2
GPU Group for Stage 1: Chunk 1 + Chunk 3
```

具体 Layer 分配取决于框架和整除条件。

## 8.4 DP 切 Batch

```
Global Batch = 8
DP = 4
```

每个 Replica：

```
B_local = 8 ÷ 4 = 2
```

所以 Replica 0 的逻辑输入：

```
X_replica0: [2,8,16]
```

四个 DP Replicas 处理不同的 2 个样本。

## 8.5 CP 切 Sequence

```
CP = 2  
S_local = 8 ÷ 2 = 4
```

每个 CP Rank:

```
X_cp_local: [2,4,16]
```

Attention 中, 本地 Query 只对应 4 个 Token:

```
Q_local: [2,4,4 Heads,4]
```

再考虑 TP=2, 每个 TP Rank 持有 2 个 Q Heads:

```
Q_local_tp: [2,4,2,4]
```

KV Heads=2, TP=2 时每 TP Rank 可持有 1 个 KV Head:

```
K/V_local_tp: [2,4,1,4]
```

但本地 Query 必须与完整 Sequence 的 KV 交互, 因此 CP Group 交换 KV, 逻辑可见长度恢复为 8:

```
K/V_visible: [2,8,1,4]
```

高性能实现可以不物化完整 AllGather, 而是以 Ring 方式逐 Chunk 计算 Attention。

## 8.6 TP 切 Dense Attention/MLP

Attention:

```
4 Q Heads ÷ TP2 = 2 Heads/Rank  
2 KV Heads ÷ TP2 = 1 KV Head/Rank
```

Dense MLP:

```
W1 global: [16,64]
W1 local:  [16,32]

W2 global: [64,16]
W2 local:  [32,16]
```

每个 TP Rank 计算部分中间维， $W_2$  后通过 AllReduce 或 ReduceScatter 归并部分和。

## 8.7 EP 切 MoE Experts

```
E = 4
EP = 2
```

每个 EP Rank:

```
2 Local Experts
```

当前 DP Replica、CP Chunk 的逻辑 Token 数:

```
Tlocal = Blocal × Slocal
        = 2 × 4
        = 8 Tokens
```

Top-2:

```
Assignments = 8 × 2 = 16
```

Router 将这 16 个 Assignments 按 Expert Owner 分组:

```
Dispatch All-to-All across EP2
→ 每 Rank 处理路由到两个 Local Experts 的 Tokens
→ Combine All-to-All
```

如果 TP 也用于 Expert 内矩阵，Expert MLP 权重还会沿 TP2 分片。Megatron 类实现中 EP+TP 通常要求启用 SP，以保持 Token/Hidden 布局可组合。

---

## 九、组合并行中一个训练 Step 的完整数据流

统一例子：B\_global=8, S=8, H=16, Layers=8, E=4, Top-2

### 1. DP4 切 Batch

每个 DP replica 获得 B\_local=8/4=2 个样本；四个 replica 处理不同数据。



### 2. CP2 切 Sequence

[2,8,16] → 每 CP rank [2,4,16]；本地 Q 长度4，KV 通过 Ring/AG覆盖完整长度8。



### 3. TP2 执行 Attention / Dense MLP

4个Q Heads → 每TP rank 2 Heads； $W_1[16,64] \rightarrow [16,32]$ ,  $W_2[64,16] \rightarrow [32,16]$ 。  
Column/Row Parallel通过 AllReduce 或 ReduceScatter/AllGather组合恢复逻辑输出。



### 4. EP2 执行 MoE Layer

E=4 → 每EP rank 2 Experts；本地逻辑Tokens=2×4=8，Top-2 → 16 Assignments。  
Dispatch All-to-All → Local Experts/Grouped GEMM → Combine All-to-All。



### 5. PP2 / VPP 传递 Stage 边界

Stage 0 Layers 0-3 → P2P Activation → Stage 1 Layers 4-7；Backward 反向传 dActivation。



### 6. Backward 后 DP / FSDP 同步模型状态

DDP：相同(tp,pp,cp,ep)分片在dp0..3间AllReduce梯度。  
FSDP：ReduceScatter梯度、分片Optimizer更新；下一次用参数前AllGather。

同一时刻并非所有通信都串行：TP/CP/EP/DP通信可在不同依赖窗口与计算重叠，但会争用SM、HBM和互联。

## 9.1 数据读取与 DP

DP Replica 0 读取样本 0-1，Replica 1 读取 2-3，依此类推。

每个 Replica 内 First PP Stage 获取 Tokens：

[2,8]

Embedding 后：

[2,8,16]

## 9.2 CP 分片

按 Sequence 切成两个 Chunk:

```
CP0: Tokens 0-3 → [2,4,16]
CP1: Tokens 4-7 → [2,4,16]
```

所有后续 Activation 主要保持 CP 分片。

## 9.3 TP Attention

每个 CP Rank 内, TP0/TP1 分别处理一半 Heads。

本地 Query:

```
[2,4,2,4]
```

CP Ring 交换 KV Chunk:

```
Round 0: 计算本地 Q × 本地 KV
Round 1: 计算本地 Q × 远端 KV
```

在线 Softmax 合并两个 KV Chunk 的 Attention 结果。

TP Output Projection 后归约部分结果, 使逻辑 Hidden 输出保持 `[2,4,16]`, 物理上可继续以 SP/TP 兼容布局保存。

## 9.4 Dense MLP 的 TP

```
[2,4,16]
→ flatten [8,16]
→ Column Parallel W1, 每TP Rank [8,32]
→ Activation
→ Row Parallel W2, 部分输出 [8,16]
→ ReduceScatter/AllReduce
```

## 9.5 MoE Layer 的 EP + TP

Router 对逻辑 `[8,16]` Token Hidden States 计算 Top-2, 得到 16 Assignments。

EP:

```
Dispatch A2A → 目标 Expert Rank
```

TP:

```
每个 Expert 内的大矩阵继续由 TP2 切分
```

本地 Experts 使用 Grouped GEMM，随后：

```
Combine A2A → Token Origin EP Rank  
→ Unpermute  
→ Weighted Sum
```

## 9.6 PP Stage 边界

Stage 0 完成 Layers 0–3 后，将 Activation 发到对应 Stage 1 Rank：

```
rank(dp,pp=0,cp,ep,tp)  
→ rank(dp,pp=1,cp,ep,tp)
```

例如：

```
rank 0 → rank 8  
rank 1 → rank 9  
...  
rank 7 → rank 15
```

通信 Tensor 保留与 CP/TP/EP 布局兼容的分片，而不是必须先在单卡上拼成全局 `[2,8,16]`。

## 9.7 Backward

反向顺序大致相反：

```
Stage 1 Backward  
→ PP P2P 发送 dActivation 给 Stage 0  
→ MoE Combine/Expert/Dispatch 的反向通信  
→ TP Collectives  
→ CP dKV ReduceScatter/Ring  
→ 各 Rank 得到本地参数梯度
```



## 9.8 DP/DDP 或 FSDP 更新

对于 Dense 参数的某个相同模型分片：

```
固定 pp,cp,ep,tp  
改变 dp
```

例如 Rank 0 对应 DP Group：

```
{0,16,32,48}
```

DDP：

```
Gradient AllReduce across DP Group  
→ 每个 Replica 的该分片参数保持一致
```

FSDP：

```
Gradient ReduceScatter  
→ 每个 DP Rank 只保留梯度分片  
→ 本地 Optimizer 更新状态分片  
→ 下次计算该 Unit 前 Parameter AllGather
```

CP Rank 之间通常复制相同权重、处理不同 Sequence Chunk，因此权重梯度还必须在适当的 DP-with-CP 组中归约。MoE Expert 参数则使用“拥有同一 Expert Shard 的 Expert-Data-Parallel Group”同步，而不是盲目跨所有 EP Rank 同步。

---

## 十、一个 Rank 到底存什么：以 Rank 0 为例

Rank 0 坐标：

```
(dp0, pp0, cp0, ep0, tp0)
```

它负责：

```
DP: Global Batch中的样本0-1  
PP: Layers 0-3  
CP: 每个样本的Sequence前4个Token  
EP: Experts 0、1  
TP: Attention Heads前半、Linear矩阵前半Shard
```

Rank 0 不拥有：

- 样本 2-7，它们属于其他 DP Replicas；
- Layers 4-7，它们属于 PP Stage 1；
- Sequence 后 4 个 Token 的长期 Activation，它们属于 CP1；
- Experts 2、3，它们属于 EP1；
- TP 的另一半矩阵/Heads，它们属于 TP Rank 1。

但在执行过程中，Rank 0 会通过通信短暂看到：

- CP1 的 KV Chunk；
- 被其他 EP Rank 路由到本地 Experts 的 Token；
- TP Collective 的部分和；
- FSDP AllGather 的临时参数；
- PP 上下游的 Activation 或 Gradient。

这就是区分 **常驻 Shard** 与 **通信时临时可见数据** 的关键。

---

## 十一、进程组与通信不会混在一起

一个物理 Rank 同时属于多个 Process Groups：

```
TP Group: 层内矩阵协作
PP Group: 相邻 Stage
CP Group: 序列上下文协作
EP Group: 专家 Token 路由
DP Group: 不同数据副本的梯度/状态同步
```

每个 Collective 必须使用正确 Group：

```
TP AllReduce 不能错误地包含 DP Rank
EP All-to-All 不能发送给不相关 PP Stage
DP AllReduce 不能混合不同 TP Shard
CP KV Exchange 只在相同模型分片位置进行
```

分布式 Hang 和 Silent Wrong Result 很多都来自 Process Group、Rank Mapping、Collective 顺序或 Tensor Shape 不一致。

---

## 十二、如何把 64 卡映射到真实硬件拓扑

假设每节点 8 张 NVLink GPU，共 8 个节点。

本例每 PP Stage 内：

$TP2 \times CP2 \times EP2 = 8 \text{ GPUs}$

一种自然映射：

一个节点 = 一个 DP Replica 的一个 PP Stage

于是：

- TP/CP/EP 通信限制在节点内 NVLink；
- PP 在节点间走较少频率的 P2P；
- DP Group 跨多个节点进行梯度同步；
- 4 个 DP Replicas  $\times$  2 个 PP Stages = 8 Nodes。

这符合通用原则：

高频、强依赖通信 (TP/EP/CP) 放快链路  
较低频或可Overlap通信 (PP/DP) 允许跨节点

但如果 EP All-to-All 的消息很大，可能需要特殊 Dispatcher 或跨节点 EP；实际布局应以节点拓扑、模型 Shape 和 Profiler 为准。

---

## 十三、组合策略的配置示例

Megatron-LM 风格的教学配置：

```
--tensor-model-parallel-size 2 \  
--pipeline-model-parallel-size 2 \  
--context-parallel-size 2 \  
--expert-model-parallel-size 2 \  
--num-experts 4 \  
--moe-router-topk 2 \  
--sequence-parallel \  
--moe-grouped-gemm \  
--moe-token-dispatcher-type alltoall \  
--data-parallel-sharding-strategy optim_grads_params \  
--use-megatron-fsdp
```

概念含义：

- TP2：切 Layer 内矩阵；
- PP2：切 Layers；
- CP2：切 Sequence；
- EP2：切 Experts；
- SP：支持 TP/EP 激活布局；
- Grouped GEMM：合并本地 Expert 小 GEMM；
- All-to-All Dispatcher：路由 Token；
- FSDP Full Shard：在 DP4 上切训练状态。

具体参数名和约束随 Megatron Core 版本变化，生产环境必须以实际版本文档和源码为准。

---

## 十四、如何选择并行配置

### 14.1 第一步：先解决显存可行性

按顺序问：

1. 单层是否放得下？放不下先增加 TP。
2. 总 Layers 是否放得下？放不下增加 PP。
3. 长序列 Activation 是否 OOM？增加 CP 或使用 Checkpoint。
4. MoE Expert 参数是否放得下？增加 EP。
5. 参数/梯度/Optimizer 是否放得下？使用 FSDP/ZeRO。

### 14.2 第二步：最小化模型并行

模型并行越大，通常通信越多、单卡 GEMM 越小。

因此在满足显存后：

尽量减小 TP、PP、CP、EP  
尽量增大 DP

但不能机械执行：MoE Expert Layer 有时增加 EP 反而比增加 TP 更高效；长序列增加 CP 可能比扩大 TP 或 Full Recompute 更好。

## 14.3 第三步：拓扑感知

- TP 优先机内；
- EP/CP 尽量机内或高速域；
- PP 适合跨节点；
- DP/FSDP 可跨节点并与 Backward Overlap；
- Process Group 必须对应物理链路。

## 14.4 第四步：用 Profiler 调整

监控：

```
Step Time / Tokens per Second / MFU
TP Collective Exposed Time
PP Bubble / Stage Imbalance
CP KV Communication
EP Dispatch/Combine A2A
DP Gradient Sync
FSDP Parameter AllGather
Per-GPU Peak Memory
GEMM Shape 与 Tensor Core Utilization
```

优化一个通信维度可能把瓶颈转移到另一个维度，所以必须端到端测量。

---

## 十五、常见误区

### 误区 1：总 GPU 数永远等于 $TP \times PP \times CP \times EP \times DP$

这是常见的笛卡尔积教学模型，也是 Megatron Guide 的基础公式；但高级 MoE Parallel Folding、Expert Tensor Parallel、Hierarchical DP 等可能让 Attention 与 MoE 使用不同进程组布局。真实 World Size 和 Group 关系应以框架配置为准。

## 误区 2: CP 和 SP 是同一个东西

---

不是。SP 是 TP 内部的激活优化，CP 是完整 Context/Sequence 维度并行，Attention 需要额外 KV 通信。

## 误区 3: FSDP 能替代 TP

---

不能。FSDP 在计算 Layer 前会 AllGather 参数；若单层本身放不下，仍需要 TP。

## 误区 4: PP 会把单层切开

---

不会。PP 切 Layers；TP 才切单层 Tensor。

## 误区 5: EP 只切参数，不通信 Token

---

错误。动态 Router 让 Token 路由到远端 Experts，Forward 通常需要 Dispatch/Combine 两次 All-to-All。

## 误区 6: 并行度越大越快

---

错误。并行度提高会减少每卡工作，但增加通信、同步和小 GEMM。最优点取决于模型 Shape 与硬件拓扑。

## 误区 7: 所有通信都必须串行执行

---

错误。DDP Bucket AllReduce、FSDP Prefetch、CP Ring、EP A2A 和 PP 调度都有 Overlap 机会；但它们可能争用 SM、HBM 与网络，不能简单相加。

---

## 十六、关键总结

---

把所有策略串成一条决策链：

先看一个 Layer:

放不下 → TP

再看模型深度:

全部 Layers 放不下 → PP / VPP

再看 Sequence Activation:

长上下文 OOM → CP

再看 MoE Experts:

全部 Experts 放不下 → EP

再看训练状态:

参数/梯度/Optimizer 放不下 → FSDP / ZeRO

模型已经能放下后:

剩余 GPU → DP 扩吞吐

组合并行不是让数据被随意切碎，而是为每种逻辑维度建立独立 Process Group:

TP 组负责层内矩阵

PP 组负责层间 Activation

CP 组负责 Context KV

EP 组负责 Expert Token

DP 组负责不同数据副本的模型一致性

一个 Rank 同时属于多个组，每个通信操作只在正确组内执行。

---

## 可选自测

### 1. World Size 与 Rank Group

配置:

TP=4, PP=2, CP=2, EP=1, DP=8

- World Size 是多少?
- 一个 Model Replica 占多少 GPU?
- 每个 DP Rank 的 Local Batch 如何由 Global Batch 计算?

### 2. Shape 题

B\_global=32, DP=4, S=32768, CP=8, H=8192,  
Q Heads=64, TP=8

计算每个 Rank 的:

- `B_local` ;
- `S_local` ;
- Q Heads 数;
- 逻辑 Q Shape;
- CP Attention 需要看到的 KV Sequence Length。

### 3. 工程选型题

一个 70B Dense 模型训练表现：

```
TP=8 已在单机
PP=8 跨8节点
DP=2
Sequence=128K
Activation OOM, 需要 Full Recomputation
TP Collective 已占 Step 20%
```

你会优先增加 TP、PP、CP 还是 DP？为什么？如何重新映射拓扑？

---

### 继续学习

1. 前置：Process Group 与 Rank Mapping —— 彻底理解多维并行如何建立通信组和避免 Collective Hang。
2. 同层对比：Megatron 3D/5D Parallel 与 DeepSpeed/FSDP —— 对比框架在状态分片和调度上的实现。
3. 下游应用：大模型并行配置自动搜索 —— 用显存模型、通信模型和拓扑约束寻找最优 TP/PP/CP/EP/DP。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

---

## 事实来源与版本边界

本文基于截至 2026-07-29 的 NVIDIA Megatron Core Parallelism Guide、Context Parallelism 文档、MoE 文档，以及 PyTorch DDP/FSDP 的稳定语义整理。不同框架在 Rank Ordering、Sequence Parallel、Expert Data Parallel、FSDP Unit、VPP Schedule、Parallel Folding 和通信后端上存在差异；具体参数、整除约束、Process Group 和性能应以实际版本源码、模型 Shape 与硬件 Profiler 为准。



